ScoreControl Backend: Progress Update

Guy Nutman · October 30, 2026

Summary. Since last week, the database went from a design doc to a live Supabase database: all 11 tables (users, projects, files, commits, branches, pull requests) are deployed with security rules, and new users get their account record automatically. We applied every schema decision from the 10/22 meeting, added automated database tests, and built Google sign-in on the backend, which only needs the Google and Supabase dashboard setup to go live. Login works end to end with Ryan's pages, pending 4 small fixes before his branch merges. Next we connect Google sign-in, then build the API routes that let users create projects, upload scores, and save versions.

Area	Status	Where
Database (11 tables, RLS, user trigger)	Live on Supabase	PR #3, merged Oct 29
10/22 schema decisions	All applied	PR #3
Google sign-in	Backend built; needs dashboard setup and button	feat/google-sign-in
Login and signup pages	Working; 4 fixes before merge	ui/ui+auth (Ryan)
Project, upload, and commit API	Next up	Not started

1. The database is live on Supabase (PR #3, deployed Oct 29)

- Tables: users, projects, project_members, blobs (file info), score_metadata, commits, commit_parents, commit_files, branches, tags, pull_requests.
- RLS is on for every table, so the public Supabase key can't read or write anything directly; all access goes through our server.
- A trigger creates a users row automatically when someone signs up or picks a username, and deleting an account removes the user and their projects.
- Oliver's username migration was already on Supabase, so login needed nothing extra.
- The tables are empty until people sign up and the project API routes exist.

2. The 10/22 schema decisions are all applied

- 25 MB per file, enforced by the database; 100 files per project, enforced in the API.
- MusicXML is the default format; MuseScore and OTHER (PDFs, audio) are also allowed.
- `parsed_ast` caches the parsed score so the diff engine doesn't re-read files.
- AI columns: `commits.ai_summary` and `pull_requests.ai_conflict_notes`. Branches can be archived (hidden without deleting).
- Two technical changes from the spec, same behaviour: no direct foreign key from users to Supabase's auth table (Prisma breaks on it), and the trigger runs as a Prisma migration so local setup works.
- Testing: 10 database tests run on every npm test, migrations were verified on a real Postgres, and all 214 tests pass.

3. Google sign-in: backend done (branch feat/google-sign-in, not pushed yet)

- A new route sends the user to Google with a one-time security check (PKCE); Oliver's existing route signs them in when they return.
- 6 tests, including that a sign-in code stolen from another browser is rejected.
- Still needed: Google Cloud OAuth client and Supabase provider setup, about 15 minutes (Guy); a "Continue with Google" button and a username form (Ryan).
- New Google users choose a username on first sign-in. A Google account with the same email as an existing password account is merged into it.

4. Login and frontend status

- Ryan's login and signup pages work end to end against the backend.
- His 4 fixes before merging: a lint error; the signup confirmation message never shows (`data.data.status`); show the API's specific error messages; read the username from `getCurrentUser()`.
- Email: Supabase's built-in sender works for team members only. A custom email provider (Resend's free tier plus a domain, about $10 a year) is needed before launch.

5. Next steps, in order

1. Finish Google sign-in: dashboard setup, Ryan's button, then merge.
2. Ryan's fixes and merge.
3. Project backend: a private Storage bucket for score files, a shared database client, and API routes to create and list projects, upload and commit, view history, download versions, and create and archive branches.
4. Later: pull requests and merges (needs the diff engine), LLM commit messages and summaries, and launch prep (email provider, hosting, production settings).

Housekeeping

- user-auth and setup are fully merged and can be deleted.
- The flaky auth test (Oliver) is still open.
- The AGENTS.md change log is up to date.